

CodeBounty — Interview Answer Sheet

How to use this: Every answer is written the way you'd actually say it out loud — short, plain, defensible. Where a question touches something from your PRD roadmap (Pattern system, AI hints, Sessions) that isn't built yet, the answer says so upfront and pivots to the design/reasoning, which is exactly what the question bank itself expects ("even though it's not built"). Fill in the [YOUR DETAIL] spots with your real specifics before the interview — those are the parts only you know.

1. Product / High-Level

What problem does this solve that LeetCode doesn't?

LeetCode gives you a huge list of problems but no structure — you solve randomly and don't build pattern recognition. CodeBounty is built to be pattern-first: every problem is tied to a technique like Sliding Window or Two Pointers, so instead of solving 500 random problems, you're deliberately building a mental toolkit. On top of that, it lets any user — not just an admin — host a timed session, so a friend group or a coding club can run a mock contest without needing a "teacher" account.

Who is the target user?

Three groups, same login: a learner prepping for placements who wants a guided path instead of a random list; a session host — could be a senior, a coding club lead, or just a friend — who wants to run a timed practice round; and a platform admin who curates the official problem bank so quality stays high.

Walk me through the user journey from signup to solving a problem.

User signs up through Clerk (GitHub OAuth), gets persisted into Postgres via Prisma on first login. They land on the problems table, filter/search for something, open a problem page which is a left/right layout — description and constraints on the left, a Monaco code editor on the right. They write code, hit Run to test against sample cases through Judge0, iterate, then Submit to run against the hidden test cases. If it's accepted, that gets tracked in their solved history and shows up on their profile stats.

What's your data model for a "session"/"contest," even though it's not built?

This is on the roadmap, not shipped yet — I want to be upfront about that. The design is: `Session` holds the metadata (title, type — PRACTICE/QUIZ/CONTEST/MOCK_INTERVIEW, start/end time, duration, visibility, a hashed join password). `SessionProblem` is the join table between a session and the existing problem bank, so a host just picks problems that already exist rather than authoring new ones. `SessionAttempt` tracks one participant's run through the session — start time, submit time, score, status. `SessionParticipant` tracks who joined and when. The key design decision: there's no "teacher" role anywhere. Anyone signed in can create a session, and whoever created it becomes the host — permissions come from ownership (`hostId` matches the current user), not from a

role flag. That keeps the whole role system down to just USER and ADMIN, where ADMIN only means “manages the official problem bank and platform,” not “can host sessions.” Hosting is a capability every user already has.

What would you build next if you had one more week?

I’d finish the Pattern system first since it’s the core differentiator — `Pattern`, `Tag`, `ProblemTag` models, a primary pattern per problem, and the `/patterns/[slug]` page with progress tracking. That’s more valuable to build before Sessions because it’s what actually makes this “more than a LeetCode clone,” per my own PRD.

2. Database / Schema

Walk me through your schema.

Core tables from the tutorial foundation: `User` (linked to Clerk via `clerkUserId`, plus role), `Problem` (title, slug, difficulty, constraints, examples, starter code per language, reference solution, test cases as JSON, plus `createdById`), `Submission` (source code, language, status, runtime, memory, linked to a user and problem), and a per-test-case result table linked to each submission. `Playlist` and a `ProblemInPlaylist` join table handle the many-to-many between playlists and problems. Roadmap additions on top of that:

`Pattern` / `Tag` / `ProblemTag` for the pattern system, and

`Session` / `SessionProblem` / `SessionAttempt` / `SessionParticipant` for hosted sessions.

Why Postgres over MongoDB for this?

The data is genuinely relational — a submission belongs to a user and a problem, a problem belongs to a pattern, a playlist has many problems and a problem can be in many playlists. That’s foreign keys and join tables, which Postgres and Prisma handle cleanly with actual referential integrity. Mongo would mean either duplicating data across documents or doing manual joins in application code — I’d be fighting the database instead of using it.

How do you handle many-to-many relationships?

Explicit join tables through Prisma. `Playlist`↔`Problem` is a `ProblemInPlaylist` table with `playlistId` and `problemId`. Same pattern for `Problem` ↔ `Tag` via `ProblemTag`. I use explicit join models rather than Prisma’s implicit many-to-many so I can attach extra fields later (like `order` or `addedAt`) without a migration that changes the relationship type.

What indexes did you add and why?

[YOUR DETAIL — list actual indexes, e.g.: unique index on `Problem.slug` for fast lookups by URL, index on `Submission.userId` + `problemId` since submission history is queried by user constantly, unique composite on `(userId, problemId)` for `SolvedProblem` so a problem can’t be marked solved twice for the same user.] The general reasoning I’d give: index whatever you filter or join on repeatedly — user-scoped queries like “my submissions” and “have I solved this” are the hottest paths in the app, so those get indexes first.

Is your schema normalized? Where did you intentionally denormalize, and why?

Mostly normalized — third normal form for the core entities. The one deliberate denormalization is storing `examples` and `test cases` as JSON columns on `Problem` instead of separate tables. I did that because test cases are always read and written as a whole unit with the problem (you never query “give me all test cases across

all problems where input contains X”), so a separate relational table would just add join overhead for no real querying benefit.

How do cascade deletes work here — what happens if you delete a User?

[YOUR DETAIL — check your actual `onDelete` behavior in `schema.prisma`.] Honest framing if you haven’t hardened this: “Right now a `User` delete would cascade to their submissions and solved records since those have a required foreign key — I haven’t tested that end-to-end, and in production I’d probably soft-delete users instead so submission history and leaderboard integrity survive account deletion.”

Why JSON columns for examples/test cases instead of separate tables?

Same answer as denormalization above — they’re always accessed together as a unit with the problem, never queried independently, and the shape varies (some problems have 2 examples, some have 5). A JSON column avoids a rigid schema for something that’s fundamentally a flexible list, without the cost of an extra join every time you load a problem page.

How would you handle a Problem needing to support a 4th, 5th language?

Starter code is already stored per-language, so schema-wise it’s just adding a key to that JSON/object rather than a migration. The real work is on the Judge0 side — mapping the new language to its Judge0 language ID, and making sure the reference solution and starter code exist for it before an admin can save the problem, since Judge0 validation happens before saving.

What’s a composite unique constraint and where do you use one?

It’s a uniqueness rule across multiple columns together rather than one column alone. I use it on `SolvedProblem` as `(userId, problemId)` — a user can only have one “solved” record per problem, not a unique `id` alone which wouldn’t stop duplicates. Same idea applies to `ProblemInPlaylist` — `(playlistId, problemId)` so you can’t add the same problem to a playlist twice.

How would you paginate at the database level instead of loading everything?

Right now the problems table loads everything and filters/sorts client-side — that’s the honest current state and it’s actually one of my known bottlenecks (see System Design section). The fix is `take / skip` (or cursor-based pagination with `cursor + take`) in the Prisma query, driven by query params in the URL, so the database only returns the page you’re viewing instead of the whole table every time.

How would you add full-text search on problem titles/descriptions?

Postgres has built-in full-text search — `tsvector / tsquery` with a GIN index, which Prisma can call via raw SQL (`$queryRaw`) since it’s not first-class in the Prisma query API. For the scale I’m at, a simpler `ILIKE '%term%'` with a trigram index (`pg_trgm`) would probably be enough and is less to maintain. I’d reach for full Postgres full-text search once search relevance actually starts mattering, not before.

3. Next.js / React

App Router vs Pages Router.

App Router, which the tutorial uses. Pages Router routes by file name in `pages/`, everything is a client component by default, and data fetching happens in `getServerSideProps` / `getStaticProps`. App Router routes by folder structure in `app/`, components are Server Components by default (rendered on the server, zero JS shipped unless you opt in), and you fetch data directly inside async Server Components instead of a separate data-fetching function.

Server Components vs Client Components — how do you decide?

Default to Server Component. I only mark something `"use client"` when it needs browser-only APIs, state, or event handlers — the Monaco code editor is the clearest example, since it has to run in the browser and respond to keystrokes. The problem description, the problems table's initial data, the profile stats — all of that can be a Server Component because it's just rendering fetched data with no interactivity of its own.

What are Server Actions, and how are they different from a traditional REST API?

A Server Action is a function marked `"use server"` that you call directly from a component like a normal function call, but it actually executes on the server — Next.js handles the network request under the hood. Compared to a REST API, I don't have to hand-write an endpoint, a fetch call, and a JSON contract for every mutation — for something like "add problem to playlist," I just call the action from the button's `onClick` /form handler. The tradeoff is it's more tightly coupled to Next.js itself; a REST API is consumable by any client, a Server Action isn't really meant to be.

How does Next.js middleware work, and what does yours protect?

Middleware runs before a request hits a route, at the edge. Mine uses Clerk's middleware to check auth state and role — it protects `/admin/*` routes so a non-admin gets redirected before the page ever renders, rather than relying on the UI just hiding the "Create Problem" button.

What's hydration, and why does it matter for a Client Component like your Monaco editor?

The server renders HTML first and sends it down, then React "hydrates" it in the browser — attaching event listeners and making it interactive. For the Monaco editor, that means the HTML shell can load fast, but the actual editor isn't typeable until hydration finishes and Monaco's JS bundle loads. That's part of why the editor is a client component with a loading state rather than something I'd try to server-render.

How would you add optimistic UI to the "solve" action?

Use `useOptimistic` (or manual state) to flip the problem's "solved" badge to green the instant Submit is clicked, before Judge0's response comes back, then reconcile — revert if the actual result comes back as failed. It makes the UI feel instant instead of waiting on Judge0's round-trip, which can take a couple seconds.

How would you cache the problems list, and how would you invalidate that cache when an admin adds a problem?

Since the problems list changes rarely (only when an admin adds/edits a problem) but is read constantly, it's a good candidate for `unstable_cache` or a tagged fetch cache in Next.js. I'd tag that cache entry (e.g.,

"problems"), and when the admin's create/edit Server Action succeeds, call `revalidateTag("problems")` so the next request rebuilds it. Submission data, by contrast, I would *not* cache the same way — it changes per-request and is user-specific.

What's the difference between `revalidatePath` and `revalidateTag`?

`revalidatePath` invalidates the cache for a specific route (e.g., `/problems`) — it's coarse, tied to a URL. `revalidateTag` invalidates every cached fetch that was tagged with that string, regardless of which route it was fetched in — it's finer-grained and works across multiple pages that might all show problem data. I'd use `revalidateTag` here because problem data shows up in more than one place (the list, the pattern page, the problem detail page) and I want all of them to go stale together.

4. Auth / Security

Why Clerk instead of building auth yourself (or using NextAuth)?

Auth is security-critical and easy to get subtly wrong — password hashing, session tokens, OAuth flows, CSRF. Clerk gives me GitHub OAuth, session management, and a middleware integration out of the box, which let me spend my time on the actual product (Judge0 integration, schema design) instead of reimplementing auth. Compared to NextAuth, Clerk also gives a hosted user-management UI, which was faster to wire up for this project.

How do you know a user is who they say they are on the server, not just the client?

Clerk's middleware verifies the session token (a signed JWT) on every server-side request — I call `auth()` inside a Server Action or route handler, and it returns the verified `userId` straight from the validated session, not from anything the client sent me. The client never gets to just claim "I'm user X"; the token is cryptographically checked server-side every time.

Why do you re-check the admin role in the API route even though the UI already hides the button?

Because the UI is not a security boundary — anyone can open dev tools, call the Server Action directly, or hit the route with curl, bypassing whatever button is or isn't rendered. The actual permission check has to happen where the mutation happens: I fetch the user's role from the database inside the Server Action itself and reject if it's not ADMIN, regardless of what the client sent.

How would you rate-limit hint requests or submission spam?

This is planned, not built yet, per the PRD. The design: a simple per-user counter with a time window — either in Postgres (a `count` + `windowStart` on the user, reset periodically) or in Redis with a sliding window / token bucket if I wanted something that doesn't hit the DB on every request. Redis would scale better since rate-limit checks are high-frequency and shouldn't add load to the primary database.

What happens to a user's data across the app if Clerk's account gets deleted?

Honestly — I haven't wired a Clerk deletion webhook yet, so right now a Clerk-side deletion wouldn't automatically clean up or anonymize the corresponding Prisma `User` row. That's a real gap I'd close before production: Clerk sends a `user.deleted` webhook, and I'd listen for it and either cascade-delete or soft-delete the app-side record depending on whether I want to preserve leaderboard/submission history.

5. Code Execution / Judge0

How does Judge0 actually run untrusted code safely?

Judge0 executes submitted code inside isolated sandboxes (built on `isolate`, a process/resource-limiting sandbox), with CPU time limits, memory limits, and no network access from inside the sandbox. It's the same category of tool real online judges use — it's specifically designed so a submission can't read the host filesystem, exhaust host resources, or reach the network, because it's untrusted code by definition.

What's the difference between "Run" and "Submit" in a typical judge, and does your app distinguish them?

Run tests against the sample/visible test cases so the user can debug their logic. Submit runs against the full hidden test case set and is what actually counts toward "solved." [YOUR DETAIL — confirm: does your app currently distinguish these as two separate actions/buttons, or is it one execution path today? Answer honestly based on what's actually built.]

Why batch submissions instead of one at a time?

Judge0 supports submitting multiple test cases in a single batch call instead of one HTTP round-trip per test case. That cuts down network overhead significantly — if a problem has 10 hidden test cases, that's 1 request instead of 10, which matters both for latency (feels faster to the user) and for not hammering the Judge0 instance with request volume.

What happens if Judge0 times out or is down — how does your app degrade?

[YOUR DETAIL — be honest about current behavior: does it show an error state, hang, retry? If there's no graceful handling yet, say so.] Honest answer if it's not handled well yet: "Right now if Judge0 doesn't respond, the user sees [current behavior] — that's not great, and what I'd add is a timeout with a clear error message telling the user to retry, plus maybe a retry-with-backoff before giving up."

How do you compare actual vs expected output — is it exact string match? What about trailing whitespace/newlines?

It's a string comparison after `.trim()` on both sides, so leading/trailing whitespace and newline differences don't cause false failures. The real limitation is that it's not semantic comparison — if expected output is `3.0` and actual is `3.00`, or a float has different precision, `.trim()` won't catch that they're logically equal, and it'd fail incorrectly. A more robust judge would parse the expected type (number, array, etc.) and compare with tolerance for floats instead of doing raw string equality.

How would you support "custom input" / arbitrary user-provided test input?

Add a mode where instead of pulling from the stored test cases, the user types their own input into a text box, and that gets sent to Judge0 the same way a normal Run does — same execution path, different source for the `stdin`. No schema changes needed since it doesn't need to be persisted as a real test case, just passed through for that one execution.

How would you scale code execution to handle 1000 concurrent submissions?

Single Judge0 instance would choke on that — the fix is running multiple Judge0 workers behind a queue instead

of calling Judge0 synchronously from the request. Submission comes in, gets written to the DB as “pending,” and pushed onto a queue (see message queue answer below); a pool of workers pulls jobs off the queue and hits Judge0, then writes the result back and the client polls or gets notified. That decouples “accept the submission” from “actually execute it,” so a burst of submissions doesn’t take down the request path.

6. System Design / Scaling

How would this scale to 100k users?

A few concrete moves, roughly in priority order: move the problems table off client-side filtering to server-side pagination with DB-level `take / skip` (this is my current known bottleneck — see below); cache problem/pattern data since it changes rarely but is read constantly; put code execution behind a queue with multiple Judge0 workers instead of one instance handling requests synchronously; add read replicas or connection pooling (PgBouncer) for Postgres once query volume grows; and add rate limiting so no single user can flood submissions or hint requests.

Where’s your current bottleneck?

Honestly, the problems table — right now filtering and pagination happen client-side after loading the full dataset, which doesn’t scale as the problem bank grows. The fix is straightforward: move filters (difficulty, pattern, search, solved status) into query params, pass them to the Prisma query, and paginate at the database level with `take / skip` so the client only ever gets one page’s worth of data.

How would you cache problem data (which changes rarely) vs submission data (which changes constantly)?

Problem/pattern data: cache aggressively with tags, invalidate only on admin writes (`revalidateTag` when an admin creates/edits a problem) — it’s read-heavy and write-rare, a textbook caching case. Submission data: don’t cache the same way — it’s written on every submit and is user-specific, so caching it would either go stale immediately or need per-user cache keys that add complexity for little benefit. I’d rely on the database being fast for that instead of caching it.

How would you horizontally scale Judge0 workers?

Run multiple Judge0 instances (or multiple Judge0 worker containers pointed at the same job queue, since Judge0 itself is built on a Redis-backed queue internally) behind a load balancer or a queue consumer pool, so submissions get distributed across instances instead of all hitting one. The application side just needs an abstraction that doesn’t hardcode a single Judge0 URL — pick a worker, or push to a queue and let the workers pull.

Would you introduce a message queue anywhere?

Yes — submission processing is the natural fit. Right now (in the tutorial-based flow) submitting code and getting a result happens more or less synchronously. A better design: “accept submission” writes a row and returns immediately, the actual Judge0 execution happens asynchronously off a queue (something like Redis-backed BullMQ, or SQS if I were on AWS), and the client polls or gets a websocket/SSE update when it’s done. That

decouples the fast, cheap part (accept + store) from the slow, resource-heavy part (execute + score), which is exactly what you want for something with unpredictable load like code execution.

7. Behavioral / Reflective

What's the hardest bug you fixed in this project?

[YOUR DETAIL — this is the one section that has to be your real story. The question bank flags two candidates: a Judge0/Windows setup issue, or a batch-size chunking issue with Judge0 submissions. Pick whichever actually happened and tell it in STAR form: what broke, why it was confusing, what you tried, what actually fixed it, what you'd do differently. A generic placeholder here will fall apart under a follow-up question, so this needs your real memory of it.]

What would you do differently if you started over?

Reasonable honest answer: I'd design the Pattern and Tag models from day one instead of bolting them on after the tutorial foundation, since pattern-first is the whole differentiator — right now it's a planned extension rather than baked into the original schema. I'd also start with server-side pagination on the problems table instead of client-side filtering, since that's the bottleneck I already know I have to fix.

What's a design decision you're not 100% happy with?

Two honest candidates from the schema: storing test case results in a way that duplicates some JSON structure between `Submission` and the per-test-case result table rather than fully normalizing it — it was faster to build but means updating in two places if the shape changes. Or: tags being handled loosely rather than as a fully relational `ProblemTag` model from day one. [YOUR DETAIL — pick whichever matches your actual current schema and go with that one; don't claim both if only one is true.]

Quick self-check before the interview

- ☐ Fill in every [YOUR DETAIL] above with your real specifics — indexes, cascade behavior, Run/Submit distinction, Judge0 failure handling, and your actual hardest-bug story.
- ☐ Re-read the honest framings out loud once — “that's on the roadmap, not built yet” and “that's a gap I'd close before production” should feel natural to say, not like an admission of failure.
- ☐ Know which pieces are tutorial-foundation (built): auth, problem CRUD, Judge0 execution, submissions, playlists, profile stats.
- ☐ Know which pieces are PRD roadmap (designed, not built): Pattern/Tag system, AI hint ladder, Sessions/contests, rate limiting, Clerk deletion webhook.